

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

PATRON ARQUITECTONICO

Se adoptará una arquitectura RESTful sobre Node.js con Express.js, siguiendo el patrón MVC simplificado (Router -> Middleware -> Controller). La metodología híbrida indica que se combinarán prácticas ágiles con fases estructuradas de diseño y seguridad.

CAPAS DE LA APLICACION

1. Capa de Entrada: Express Router que expone GET /hola
2. Capa de Seguridad (Middlewares en cadena): CORS, Rate Limiting, JWT Validator, Input Validator, Access Logger
3. Capa de Negocio: Controller que construye la respuesta JSON
4. Capa de Documentacion: Swagger UI integrado en /api-docs

SEGURIDAD (DevSecOps)

- JWT: Middleware de verificación de token en cada request protegido. Se usará jsonwebtoken. El token debe enviarse en el header Authorization: Bearer <token>.
- CORS: Configuración restrictiva con whitelist de dominios permitidos, definida por variables de entorno.
- Rate Limiting: Uso de express-rate-limit para limitar requests por IP (ej. 100 req/15min).
- Validación de Entrada: Uso de express-validator para sanitizar y validar headers y query params.
- Helmet: Middleware de seguridad HTTP para cabeceras seguras (XSS, HSTS, etc.).
- Access Logger: Uso de morgan combinado con un logger estructurado (winston) para registrar cada solicitud con timestamp, IP, método, ruta y status code.
- Variables de entorno: Gestión de secretos (JWT_SECRET, PORT, ALLOWED_ORIGINS) mediante dotenv, nunca hardcodedos.

TECNOLOGIAS

- Runtime: Node.js 20 LTS
- Framework: Express.js 4.x
- Autenticacion: jsonwebtoken
- Seguridad HTTP: helmet
- CORS: cors
- Rate Limiting: express-rate-limit
- Validacion: express-validator
- Logging: morgan + winston
- Documentacion: swagger-jsdoc + swagger-ui-express
- Testing: Jest + Supertest
- Linting: ESLint + Prettier
- Variables de entorno: dotenv

ESTRATEGIA DE TESTING

- Tests unitarios para middlewares y controllers
- Tests de integración para el endpoint GET /hola con y sin token válido
- Cobertura mínima esperada: 80%

JIRA: XP-9 | Metodología: Hybrid

2. MÉTRICAS DE ITERACIÓN (QA)

- src/server.js | Estado: APROBADO | Iteraciones: 1
- src/config/app.js | Estado: APROBADO | Iteraciones: 1
- src/config/cors.js | Estado: APROBADO | Iteraciones: 1
- src/config/swagger.js | Estado: APROBADO | Iteraciones: 1

- src/config/logger.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/auth.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/rateLimiter.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/inputValidator.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/middlewares/accessLogger.middleware.js | Estado: APROBADO | Iteraciones: 1
- src/routes/hola.routes.js | Estado: APROBADO | Iteraciones: 1
- src/controllers/hola.controller.js | Estado: APROBADO | Iteraciones: 1
- src/utils/responseBuilder.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/hola.controller.test.js | Estado: APROBADO | Iteraciones: 1
- tests/unit/auth.middleware.test.js | Estado: APROBADO | Iteraciones: 1
- tests/integration/hola.routes.test.js | Estado: APROBADO | Iteraciones: 1
- logs/.gitkeep | Estado: WARNING (Forzado) | Iteraciones: 3
- .env.example | Estado: APROBADO | Iteraciones: 1
- .gitignore | Estado: APROBADO | Iteraciones: 1
- .eslintrc.js | Estado: APROBADO | Iteraciones: 1
- .prettierrc | Estado: APROBADO | Iteraciones: 1
- jest.config.js | Estado: APROBADO | Iteraciones: 1
- package.json | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: APROBADO | Iteraciones: 1